a) Realice la elección de un diseño para esta realidad. Debe indicar los Tipos Abstractos de Datos elegidos, por ejemplo: Vehículos = Secuencia (Vehículo)

Vehículo = matricula X marca X año

Auto = cant_puertas X precio_base X Vehículo

Moto = cilindrada X precio_base X Vehículo

Observaciones:

-
-
-
-

b) Realice el correspondiente desglose de requerimientos. Requerimiento 1a: Alta de vehículo

- Entrada: Datos del vehículo (matrícula, marca, año, y específicos según si es Auto o Moto).
- Salida: Mensaje de éxito y vehículo ingresado, o Mensaje de error.
- Proceso:
    1. Se busca en la secuencia de vehículos si ya existe uno con la matrícula ingresada.
    2. Si la matrícula ya existe:
        - Se retorna un error indicando "La matrícula ya existe".
    3. Sino:
        - Se crea la instancia correspondiente (Auto o Moto) y se ingresa a la secuencia del sistema.

Requerimiento 1b: Baja de vehículo

- Entrada: Matrícula (String).
- Salida: Mensaje de confirmación de baja o Error si no existe.
- Proceso:
    1. Se busca en la secuencia el vehículo correspondiente a la matrícula ingresada.
    2. Si el vehículo existe:

        - Se elimina de la secuencia de vehículos del sistema.
        - Se retorna un mensaje de éxito.
    3. Sino:
        - Se retorna un error: "Vehículo no encontrado".

Requerimiento 2: Calcular suma de precios de alquiler diario

- Entrada: Ninguna (opera sobre la colección actual).
- Salida: Suma total de los precios (double).
- Proceso:
    1. Se inicializa un acumulador totalSuma = 0.0.
    2. Se obtiene el iterador de la secuencia de vehículos para recorrerlos uno a uno.
    3. Por cada vehículo en la secuencia:
        - Se invoca de forma polimorférica al método precioAlquilerDiario().
        - Se suma el resultado al acumulador totalSuma.
    4. Se devuelve el valor de totalSuma.

Requerimiento 3: Obtener vehículos con mayor y menor precio de alquiler diario

- Entrada: Ninguna.
- Salida: Datos del vehículo con mayor precio y datos del vehículo con menor precio (String).
- Proceso:
    1. Si la secuencia de vehículos está vacía, se retorna un mensaje indicando que no hay datos.
    2. Se inicializan dos variables para registrar los extremos (vehiculoMayor y vehiculoMenor) con el primer elemento de la secuencia.
    3. Se recorre el resto de la secuencia comparando el precioAlquilerDiario() de cada vehículo:
        - Si el precio actual es mayor que el de vehiculoMayor, se actualiza dicha variable.
        - Si el precio actual es menor que el de vehiculoMenor, se actualiza dicha variable.

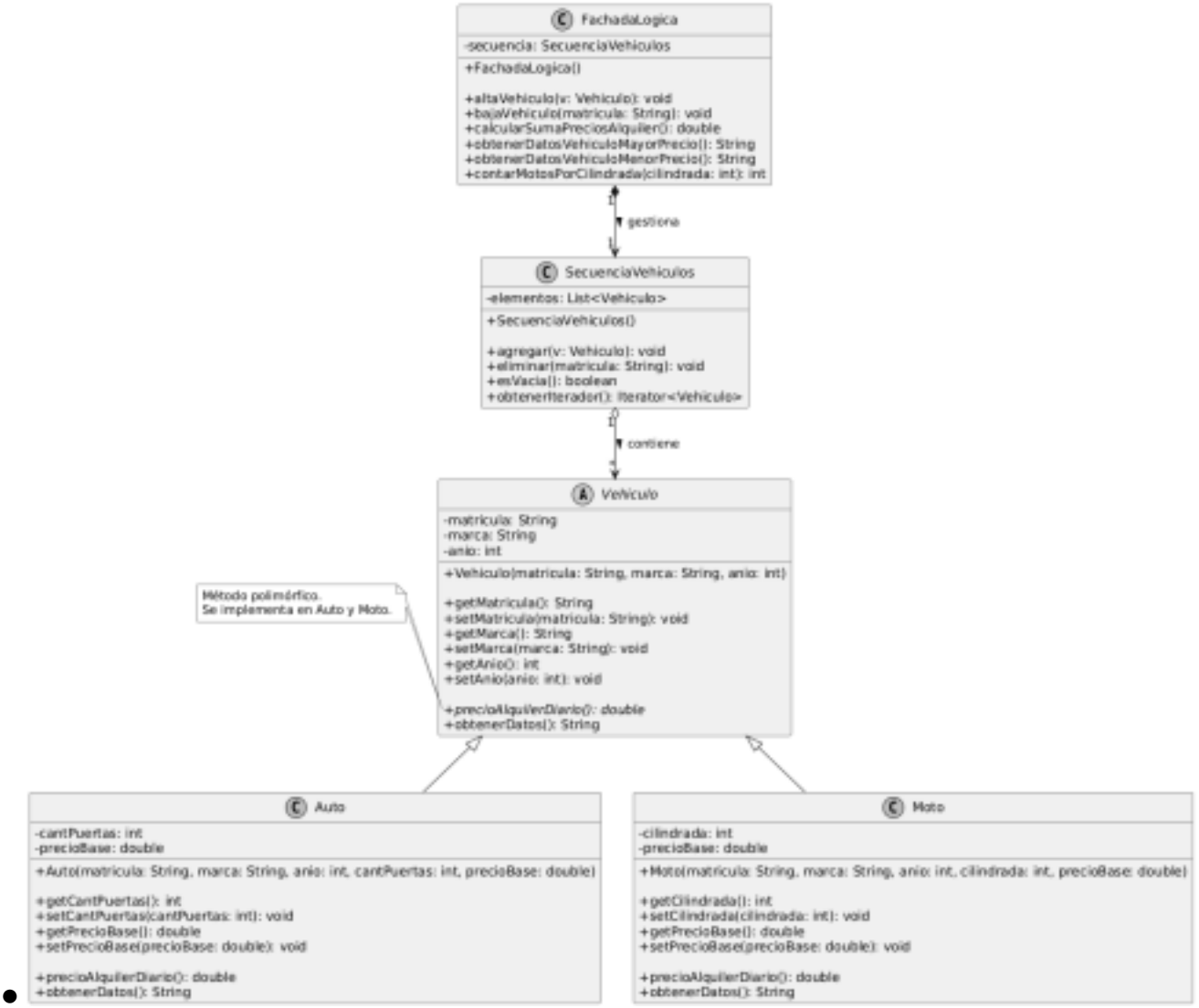
4. Al finalizar el recorrido, se invocan los métodos obtenerDatos() de ambos vehículos encontrados y se retornan las cadenas de texto correspondientes.

Requerimiento 4: Contar motos por cilindrada

- Entrada: Cilindrada a buscar (int).
- Salida: Cantidad de motos que coinciden con esa cilindrada (int).
- Proceso:
    1. Se inicializa un contador cantidadMotos = 0.
    2. Se recorre la secuencia de vehículos utilizando el iterador.
    3. Por cada elemento:
        - Se verifica si el objeto actual es una instancia de la clase Moto.
        - Si es Moto: Se evalúa mediante getCilindrada() si coincide con la cilindrada recibida como entrada.
            - Si coincide, se incrementa cantidadMotos en 1.
    4. Se retorna el valor final de

c) En base a las partes anteriores, realice la especificación del diseño elegido.

Debe presentar el Diagrama de Clases de Implementación en UML, mostrando solamente los atributos, y luego detallar la especificación de cada clase por separado, incluyendo:


FachadaLogica
-secuencia: SecuenciaVehiculos
+FachadaLogica()
+altaVehiculo(v: Vehiculo): void
+bajaVehiculo(matricula: String): void
+calcularSumaPreciosAlquiler(): double
+obtenerDatosVehiculoMayorPrecio(): String
+obtenerDatosVehiculoMenorPrecio(): String
+contarMotosPorCilindrada(cilindrada: int): int
1
gestiona
1
SecuenciaVehiculos
-elementos: List<Vehiculo>
+SecuenciaVehiculos()
+agregar(v: Vehiculo): void
+eliminar(matricula: String): void
+esVacia(): boolean
+obtenerIterador(): Iterator<Vehiculo>
1
contiene
*
Vehiculo
-matricula: String
-marca: String
-anio: int
+Vehiculo(matricula: String, marca: String, anio: int)
+getMatricula(): String
+setMatricula(matricula: String): void
+getMarca(): String
+setMarca(marca: String): void
+getAnio(): int
+setAnio(anio: int): void
+precioAlquilerDiario(): double
+obtenerDatos(): String
Método polimórfico.
Se implementa en Auto y Moto.
Auto
-cantPuertas: int
-precioBase: double
+Auto(matricula: String, marca: String, anio: int, cantPuertas: int, precioBase: double)
+getCantPuertas(): int
+setCantPuertas(cantPuertas: int): void
+getPrecioBase(): double
+setPrecioBase(precioBase: double): void
+precioAlquilerDiario(): double
+obtenerDatos(): String
Moto
-cilindrada: int
-precioBase: double
+Moto(matricula: String, marca: String, anio: int, cilindrada: int, precioBase: double)
+getCilindrada(): int
+setCilindrada(cilindrada: int): void
+getPrecioBase(): double
+setPrecioBase(precioBase: double): void
+precioAlquilerDiario(): double
+obtenerDatos(): String